UNIVERSIDAD ROSARIO CASTELLANOS
DIRECCIÓN DE SISTEMAS Y TECNOLOGÍA • ARQUITECTURA DE SOFTWARE NoSQL

GUÍA MAESTRA DEL LENGUAJE MQL (MONGODB QUERY LANGUAGE) EN EL SISTEMA CAFÉ URC EXPRESS

Ubicación Exacta en el Código Fuente, Traducción de Django ORM a MQL Nativo, Guía Operativa de Actualización, Modificación, Eliminación (CRUD) y Manipulación de Subdocumentos Embebidos

Base de Datos NoSQL:	MongoDB (Base de datos: 'cafeteria')
Controlador / Driver:	django-mongodb-backend 6.1.0 + PyMongo 4.18.2
Framework Backend:	Django 6.1 (Python 3.12+ / 3.14)
Colecciones Activas:	users, categories, products, inventory, orders, shifts
Subdocumentos Embebidos:	DetalleItem (Array), PagoMercadoPago (Objeto inmutable)
Enfoque de la Guía:	Localización de código, Actualizar ($set, $inc), Modificar, Borrar ($unset, deleteOne/Many) y MQL Nativo

🍃 1. Fundamentos: ¿Qué es el Lenguaje MQL y su Rol en el Proyecto?

MQL (MongoDB Query Language) es el lenguaje nativo de consulta, manipulación y agregación de MongoDB. A diferencia del SQL tradicional que opera sobre tablas relacionales compuestas de filas y columnas rígidas mediante sentencias como SELECT, INSERT INTO, UPDATE o DELETE, MQL opera directamente sobre documentos BSON (Binary JSON) y colecciones estructuradas de manera flexible mediante sintaxis de objetos y operadores especializados ($set, $inc, $in, $push, etc.).

Los 3 Niveles de MQL en este Proyecto: En el proyecto Café URC Express, el lenguaje MQL coexiste en tres niveles de interacción:

- **1. Nivel ORM (Traductor Automático Python -> MQL):** Nivel donde tú escribes código en Python utilizando modelos de Django (Pedido, Producto, Inventario). El paquete oficial django-mongodb-backend (versión 6.1.0) intercepta cada llamada de Python y la compila transparentemente en comandos de MQL que son transmitidos a MongoDB a través de sockets TCP.
- **2. Nivel Driver Nativo (PyMongo):** El driver oficial de MongoDB en Python instalado en tu entorno virtual (pymongo 4.18.2). Permite enviar comandos MQL nativos directos (como db.orders.find_one(), update_one(), delete_many()) desde scripts de Python sin pasar por el ORM de Django.
- **3. Nivel Shell y GUI (mongosh & MongoDB Compass):** La consola de comandos oficial (mongosh) o la interfaz gráfica MongoDB Compass. Aquí es donde se ejecuta MQL en su forma más pura y directa (ej. db.orders.updateOne({_id: ObjectId('...')}, {$set: {estado: 'DELIVERED'}})).

> 🔌 **DÓNDE SE CONFIGURA LA CONEXIÓN MONGODB**
> La conexión principal a MongoDB se encuentra declarada en el archivo config/settings.py (líneas 77-88). Allí se define el motor 'django_mongodb_backend', el host 'mongodb://localhost:27017/' y el nombre de la base de datos 'cafeteria'. Todas las operaciones de la cafetería impactan directamente en esta base de datos.

📂 2. Localización Exacta: ¿Dónde está el Código MQL en el Proyecto?

Si buscas sentencias de texto plano tipo SQL (SELECT * FROM...) no las encontrarás, porque este proyecto es 100% NoSQL. A continuación se detalla exactamente en qué archivos y funciones se ejecutan las operaciones del lenguaje MQL:

Archivo Fuente	Líneas / Módulo	Operación MQL Ejecutada	Propósito en la Cafetería
miapp/models.py	Líneas 235-250 (Inventario)	updateOne ($set / $inc)	Descuenta stock en venta y reabastece existencias físicas.
miapp/models.py	Líneas 400-415 (TurnoCorte)	updateOne ($set)	Cierre de turno de caja y cálculo de arqueo ciego.
miapp/views.py	Líneas 169-202 (menu_cliente)	find / find_one	Consulta catálogo activo, categorías y stock para la PWA.
miapp/views.py	Líneas 490-503 (api_crear_pedido)	insertOne (con Array)	Inserta un pedido nuevo con ítems embebidos y PIN seguro.
miapp/views.py	Líneas 561-570 (webhook MP)	updateOne ($set) find_one	Actualiza estatus de pago a APPROVED y descuenta inventario.
miapp/views.py	Líneas 747-748	updateOne ($set)	Transición de comanda: PAID -> IN_PREP -

	(api_estado_kds)		> READY -> DELIVERED.

➕ 3. Inserción de Datos: Dónde y Cómo se Crea en MQL (Insert)

En MongoDB, la creación de nuevos registros se realiza mediante los comandos MQL insertOne() (para un documento) o insertMany() (para inserciones en lote). En el código de Django, esto se detona mediante el método .objects.create() o instanciando el modelo y ejecutando .save().

3.1 Inserción de un Pedido con Subdocumentos Embebidos (PWA Cliente)

📍 **Dónde está:** Ubicación en el código: miapp/views.py -> función api_crear_pedido_cliente (líneas 490 a 503).

Cuando el estudiante o docente realiza una compra desde su celular, la función recibe los productos, genera el número de turno (#URC-XXX), el código PIN de retiro seguro, los ítems embebidos inmutables (DetalleItem) y el subdocumento inicial de pago (PagoMercadoPago).

Código Python en tu Proyecto (miapp/views.py):

```
// --- Python (Django ORM en miapp/views.py) ---
# 1. Instanciar subdocumentos embebidos
detalles_embebidos = [
    DetalleItem(producto_id=str(prod.id), sku=prod.sku, nombre_prod=prod.nombre,
                precio_unitario=prod.precio_venta, cantidad=2, subtotal=116.00,
                area_despacho='Cocina', notas='Bien dorada, con rajas')
]
pago_mp = PagoMercadoPago(mp_payment_id='MP-20261022-99', metodo_pago='QR_MERCADOPAGO',
                          estatus='PENDING', monto=116.00)

# 2. Inserción del documento principal en la colección 'orders'
pedido = Pedido.objects.create(
    order_number='#URC-105',
    cliente=cliente_perfil,
    cliente_nombre='Valeria Gómez Sánchez',
    codigo_pin='PIN-412',
    estado='PENDING_PAYMENT',
    tipo_despacho='Cocina Caliente',
    items=detalles_embebidos,
    subtotal=Decimal('116.00'),
    total=Decimal('116.00'),
    pago=pago_mp,
    origen='PWA Móvil',
    tiempo_espera_estimado=7
)
```

Comando MQL Nativo que se Ejecuta en MongoDB:

```
// --- MQL Nativo (db.orders.insertOne) ---
use cafeteria;

db.orders.insertOne({
  order_number: "#URC-105",
```

```
  cliente_nombre: "Valeria Gómez Sánchez",
  codigo_pin: "PIN-412",
  estado: "PENDING_PAYMENT",
  tipo_despacho: "Cocina Caliente",
  items: [
    {
      producto_id: "6717a1b2c3d4e5f6a7b8c9d0",
      sku: "URC-TRT-01",
      nombre_prod: "Torta de Milanesa con Queso",
      precio_unitario: NumberDecimal("58.00"),
      cantidad: 2,
      subtotal: NumberDecimal("116.00"),
      area_despacho: "Cocina",
      notas: "Bien dorada, con rajas"
    }
  ],
  pago: {
    mp_payment_id: "MP-20261022-99",
    metodo_pago: "QR_MERCADOPAGO",
    estatus: "PENDING",
    monto: NumberDecimal("116.00"),
    fecha_pago: null
  },
  subtotal: NumberDecimal("116.00"),
  total: NumberDecimal("116.00"),
  origen: "PWA Móvil",
  tiempo_espera_estimado: 7,
  creado_en: ISODate("2026-10-22T10:35:00Z"),
  actualizado_en: ISODate("2026-10-22T10:35:00Z")
});
```

🔍 4. Consultas y Filtros: Dónde y Cómo se Busca en MQL (Find)

Para leer datos, MQL utiliza db.coleccion.find(filtro, proyeccion) o db.coleccion.findOne(filtro). En Django, esto corresponde a .objects.filter(), .objects.get() y .objects.all().

4.1 Consulta de Pedidos Activos en Pantalla KDS (Cocina y Mostrador)

📍 **Dónde está:** Ubicación en el código: miapp/views.py -> función pantalla_kds (líneas 295 a 305).

La pantalla de cocina necesita únicamente las órdenes que ya están pagadas (PAID) o en preparación (IN_PREPARATION), ordenadas cronológicamente para atender primero a quienes llegaron primero (FIFO).

Código Python en tu Proyecto:

```
// --- Python (Django ORM) ---
# Consulta en Django ORM con operador __in y orden ascendente
pedidos_activos = Pedido.objects.filter(
    estado__in=['PAID', 'IN_PREPARATION']
).order_by('creado_en')
```

Comando MQL Nativo Equivalente:

```
// --- MQL Nativo (db.orders.find con $in) ---
// MQL: Operador $in para múltiples estados y sort: 1 (ascendente)
db.orders.find({
  estado: { $in: ["PAID", "IN_PREPARATION"] }
}).sort({ creado_en: 1 });
```

4.2 Consulta con Notación de Punto (Dot Notation) para Subdocumentos

MongoDB permite buscar dentro de objetos anidados sin necesidad de costosos 'JOIN'. Por ejemplo, para buscar todos los pedidos pagados mediante Mercado Pago con tarjeta de débito o QR:

```
// --- MQL Nativo (Dot Notation sobre Subdocumentos) ---
// MQL: Búsqueda sobre el subdocumento 'pago'
db.orders.find({
  "pago.estatus": "APPROVED",
  "pago.metodo_pago": "QR_MERCADOPAGO"
});
```

📝 5. Modificación y Actualización: Dónde y Cómo se Cambia en MQL (Update)

Esta es una de las partes más críticas del sistema. En MQL, NUNCA se sobreescribe un documento completo para cambiar solo un campo; se utilizan operadores de actualización atómicos como $set (para asignar valores), $inc (para sumar o restar cantidades numéricas como stock), $push (para agregar a un array) o $currentDate.

5.1 Caso A: Cambio de Estado de Comanda en Pantalla KDS (Cocina)

📍 **Dónde está:** Ubicación en el código: miapp/views.py -> función api_actualizar_estado_kds (líneas 747 a 749).

Cuando el cocinero hace clic en 'Listo para Retiro', el pedido cambia de 'IN_PREPARATION' a 'READY_FOR_PICKUP'. En Django se ejecuta pedido.estado = nuevo_estado y pedido.save().

Código Python en tu Proyecto:

```
// --- Python (Django ORM) ---
pedido = Pedido.objects.filter(id=order_id).first()
pedido.estado = 'READY_FOR_PICKUP'
pedido.save()
```

Comando MQL Nativo que se Ejecuta:

```
// --- MQL Nativo (db.orders.updateOne con $set) ---
// MQL: updateOne con operador atómico $set
db.orders.updateOne(
  { _id: ObjectId("6717a1b2c3d4e5f6a7b8c9d0") },
  {
    $set: {
```

```
      estado: "READY_FOR_PICKUP",
      actualizado_en: new Date()
    }
  }
);
```

5.2 Caso B: Descuento Atómico de Inventario al Vender (Blindaje contra Sobreventas)

Dónde está: Ubicación en el código: miapp/models.py -> método descontar() de la clase Inventario (líneas 235 a 241).

Al confirmarse una venta por mostrador o por Mercado Pago, el stock debe disminuir de forma segura.

Código Python en tu Proyecto:

```
// --- Python (Método en Inventario) ---
# miapp/models.py (Línea 238)
def descontar(self, cantidad):
    if self.stock_actual >= cantidad:
        self.stock_actual -= cantidad
        self.save(update_fields=['stock_actual', 'actualizado_en'])
        return True
    return False
```

Comando MQL Nativo (Uso del Operador $inc):

```
// --- MQL Nativo (Operador $inc en db.inventory) ---
// MQL: Decrementar el campo 'stock_actual' en 2 unidades de forma atómica
db.inventory.updateOne(
  { producto_id: ObjectId("6717a1b2c3d4e5f6a7b8c9d0") },
  {
    $inc: { stock_actual: -2 },
    $set: { actualizado_en: new Date() }
  }
);
```

5.3 Caso C: Actualización Masiva de Turnos Previos (Cierre de Caja)

Dónde está: Ubicación en el código: miapp/views.py -> función api_abrir_turno (línea 819).

Al abrir un nuevo turno, el sistema cierra automáticamente cualquier turno anterior que haya quedado en estado 'ABIERTO'.

```
// --- Python vs MQL (updateMany) ---
# Python (Django ORM):
TurnoCorte.objects.filter(cajero=perfil, estado='ABIERTO').update(estado='CERRADO',
fecha_cierre=timezone.now())

// MQL Nativo equivalente (updateMany):
db.shifts.updateMany(
  { cajero_id: ObjectId("..."), estado: "ABIERTO" },
  {
    $set: {
```

```
      estado: "CERRADO",
      fecha_cierre: new Date()
    }
  }
);
```

🗑️ 6. Eliminación de Datos: Dónde y Cómo se Borra en MQL (Delete)

En el lenguaje MQL de MongoDB, el borrado físico de documentos se lleva a cabo mediante los métodos db.coleccion.deleteOne(filtro) y db.coleccion.deleteMany(filtro). Si se desea eliminar una colección entera, se utiliza db.coleccion.drop().

6.1 Borrado Físico (Hard Delete) en Django Admin y Código

📍 **Dónde está:** Ubicación en el código: miapp/admin.py (líneas 72 a 75) y método nativo .delete() de Django.

En el panel de administración (/admin), los administradores pueden seleccionar productos, categorías o turnos y eliminarlos. Cuando ejecutas obj.delete() en Python, Django envía a MongoDB un comando deleteOne().

Código Python en Django:

```
// --- Python (Django ORM .delete()) ---
# Eliminar un producto específico por su SKU
producto = Producto.objects.filter(sku='URC-TRT-01').first()
if producto:
    producto.delete()

# Eliminar pedidos cancelados con más de 30 días de antigüedad
Pedido.objects.filter(estado='CANCELLED').delete()
```

Comandos MQL Nativos Equivalentes:

```
// --- MQL Nativo (deleteOne / deleteMany) ---
// 1. Borrar un solo documento por filtro exacto:
db.products.deleteOne({ sku: "URC-TRT-01" });

// 2. Borrar múltiples documentos que coincidan con un criterio:
db.orders.deleteMany({ estado: "CANCELLED" });

// 3. Borrar un documento por su clave primaria _id:
db.orders.deleteOne({ _id: ObjectId("6717a1b2c3d4e5f6a7b8c9d0") });
```

> ⚠ **REGLA DE ORO DE ARQUITECTURA NoSQL: BORRADO LÓGICO VS FÍSICO**
> En un sistema transaccional de cafetería universitaria, los pedidos NUNCA deben borrarse físicamente con deleteOne() porque se perdería la auditoría de caja (Arqueo Ciego) y las estadísticas contables del corte Z. En su lugar, el sistema implementa BORRADO LÓGICO (Soft Delete): se ejecuta un updateOne() cambiando el campo estado a 'CANCELLED' o activo a false en productos (db.products.updateOne({sku: '...'}, {$set: {activo: false}})). De este modo, el ítem deja de verse en el menú pero permanece en el historial para auditorías.

🧩 7. Manejo Avanzado de Subdocumentos y Arreglos en MQL

Uno de los mayores atractivos de MongoDB frente a SQL es su capacidad de almacenar estructuras jerárquicas dentro del mismo documento. En este proyecto tenemos dos campos embebidos fundamentales en la colección 'orders':

- **1. Objeto Embebido 'pago' (PagoMercadoPago):** Subdocumento individual con los datos de pasarela (mp_payment_id, metodo_pago, estatus, monto, fecha_pago).
- **2. Arreglo Embebido 'items' (DetalleItem):** Arreglo de subdocumentos con la instantánea inmutable de los productos comprados (producto_id, sku, precio_unitario, cantidad, notas).

Operaciones MQL sobre Arreglos y Subdocumentos:

A. Buscar Pedidos que Contengan un Producto Específico dentro del Arreglo:

```
// --- MQL Nativo ($elemMatch sobre arreglo de ítems) ---
// MQL: Operador $elemMatch para buscar dentro del arreglo 'items'
db.orders.find({
  items: {
    $elemMatch: {
      sku: "URC-TRT-01",
      area_despacho: "Cocina"
    }
  }
});
```

B. Agregar un Nuevo Ítem al Arreglo Embebido sin Reescribir el Pedido ($push):

```
// --- MQL Nativo ($push para agregar subdocumento) ---
// MQL: Operador $push para insertar un nuevo ítem a la lista de items
db.orders.updateOne(
  { order_number: "#URC-105" },
  {
    $push: {
      items: {
        producto_id: "6717a1b2c3d4e5f6a7b8c9d1",
        sku: "URC-BEB-01",
        nombre_prod: "Agua Purificada 600ml",
        precio_unitario: NumberDecimal("15.00"),
        cantidad: 1,
        subtotal: NumberDecimal("15.00"),
        area_despacho: "Mostrador",
        notas: "Fría"
      }
    },
    $inc: { total: NumberDecimal("15.00") }
  }
);
```

💻 8. Guía Práctica: ¿Cómo Ejecutar Comandos MQL en tu Computadora?

Tienes 3 maneras directas de interactuar con el lenguaje MQL sobre la base de datos 'cafeteria' de este proyecto:

Método 1: Desde la Consola mongosh (MongoDB Shell)

Abre tu terminal de Windows (PowerShell o CMD) y escribe:

```
// --- Comandos mongosh en Terminal ---
# 1. Abrir mongosh
mongosh

# 2. Conectarse a la base de datos del proyecto
use cafeteria;

# 3. Ver todas las colecciones
show collections;

# 4. Contar pedidos existentes
db.orders.countDocuments();

# 5. Ver el último pedido registrado de forma legible
db.orders.find().sort({ creado_en: -1 }).limit(1).pretty();
```

Método 2: Desde MongoDB Compass (Interfaz Gráfica)

Abre MongoDB Compass y conéctate a mongodb://localhost:27017. Selecciona la base de datos 'cafeteria' y haz clic en la colección 'orders'. En la barra superior de filtro ('Filter'), puedes escribir MQL directo como:

- { estado: 'READY_FOR_PICKUP' } -> Para ver los pedidos listos para entrega.
- { 'pago.metodo_pago': 'QR_MERCADOPAGO' } -> Para filtrar por ventas con código QR.
- { total: { $gte: 100 } } -> Para ver compras mayores o iguales a $100 MXN.

Método 3: Ejecutar MQL Nativo desde Python en tu Proyecto (vía PyMongo)

Si necesitas ejecutar una consulta MQL directa en un script o vista de Django sin restricciones del ORM, puedes usar el cliente directo de PyMongo que ya está configurado:

```
// --- Python con PyMongo Directo ---
from pymongo import MongoClient

# Conexión directa al servidor local
client = MongoClient('mongodb://localhost:27017/')
db = client['cafeteria']

# 1. MQL nativo: Búsqueda
pedido = db.orders.find_one({'order_number': '#URC-101'})
print('Cliente:', pedido['cliente_nombre'])

# 2. MQL nativo: Actualización con $set
db.orders.update_one(
```

```
    {'order_number': '#URC-101'},
    {'$set': {'estado': 'DELIVERED'}}
)

# 3. MQL nativo: Borrado con delete_one
db.orders.delete_one({'order_number': '#URC-999'})
```

9. Tabla de Equivalencias Rápidas: Django ORM <---> MQL Nativo

Operación Deseada	Sintaxis Django ORM (Python)	Sintaxis MQL Nativa (MongoDB)
Insertar un documento	`Pedido.objects.create(campo='val')`	`db.orders.insertOne({ campo: 'val' })`
Buscar todos	`Producto.objects.all()`	`db.products.find({})`
Buscar con filtro simple	`Producto.objects.filter(activo=True)`	`db.products.find({ activo: true })`
Buscar por ID único	`Pedido.objects.get(id=mi_id)`	`db.orders.findOne({ _id: ObjectId(mi_id) })`
Buscar con 'IN' (múltiples)	`Pedido.objects.filter(estado__in=['A','B'])`	`db.orders.find({ estado: { $in: ['A','B'] } })`
Actualizar un solo objeto	`pedido.estado = 'X'` `pedido.save()`	`db.orders.updateOne({ _id: id }, { $set: { estado: 'X' } })`
Actualizar en lote (Bulk)	`Turno.objects.filter(e='A').update(e='C')`	`db.shifts.updateMany({ e: 'A' }, { $set: { e: 'C' } })`
Incrementar / Decrementar	`inv.stock_actual -= 2` `inv.save()`	`db.inventory.updateOne({}, { $inc: { stock_actual: -2 } })`
Borrar documento(s)	`pedido.delete()` `Pedido.objects.filter(e='C').delete()`	`db.orders.deleteOne({ _id: id })` `db.orders.deleteMany({ e: 'C' })`

Conclusión de Arquitectura: En resumen: MQL no es un archivo único en el proyecto, sino el lenguaje en el que 'habla' la base de datos MongoDB. Tu código en Python utiliza Django ORM como capa de abstracción para generar MQL de forma limpia y tipada, pero cada vez que ejecutas .create(), .filter(), .save(), .update() o .delete(), se ejecuta exactamente la operación MQL correspondiente descrita en esta guía.